

**Question**

Write Java programs to demonstrate common interfaces in the Java Collection Framework, including collections, lists, sets, queues, maps, and iterators.

**Steps for Execution**

1. Open your IDE (Eclipse/VS Code) or Command Prompt.
2. Save the selected program in a Java file with the same name as its public class.
3. Write or paste the complete program code into the file.
4. Save the file.
5. Compile the program using `javac ClassName.java`.
6. Run it using `java ClassName`.
7. Observe the output and repeat for the other programs.

**Algorithm**

1. Start the program.
2. Create an implementation of the interface demonstrated in the selected example.
3. Add sample values and perform the interface operations.
4. Display the values and results.
5. Stop the program.

## UML Class Diagram

Each example is a separate Java class with its own main method.

|                                                                                  |                                  |
|----------------------------------------------------------------------------------|----------------------------------|
| <b>CollectionInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args)   | <b>ListInterfaceDemo</b>         |
| <b>SetInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args)          | <b>SortedSetInterfaceDemo</b>    |
| <b>NavigableSetInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args) | <b>QueueInterfaceDemo</b>        |
| <b>DequeInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args)        | <b>MapInterfaceDemo</b>          |
| <b>SortedMapInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args)    | <b>NavigableMapInterfaceDemo</b> |
| <b>IteratorInterfaceDemo</b><br>+main(String[] args)<br>+main(String[] args)     | <b>ListIteratorInterfaceDemo</b> |

## CODE: SRC

### Program 1: Collection Interface Demo

#### Class: CollectionInterfaceDemo

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;

public class CollectionInterfaceDemo {
    public static void main(String[] args) {

        Collection<Integer> c = new ArrayList<>();

        // add()
        c.add(10);
        c.add(20);
        c.add(30);
        System.out.println("After add: " + c);

        // addAll()
        Collection<Integer> c2 = new ArrayList<>();
        c2.add(40);
        c2.add(50);

        c.addAll(c2);
        System.out.println("After addAll: " + c);

        // remove()
        c.remove(20);
        System.out.println("After remove: " + c);

        // contains()
        System.out.println("Contains 30: " + c.contains(30));

        // containsAll()
        System.out.println("Contains all c2: " + c.containsAll(c2));

        // size()
        System.out.println("Size: " + c.size());

        // isEmpty()
        System.out.println("Is empty: " + c.isEmpty());

        // iterator()
        System.out.print("Elements using Iterator: ");
        Iterator<Integer> it = c.iterator();

        while (it.hasNext()) {
            System.out.print(it.next() + " ");
        }

        System.out.println();

        // removeAll()
        c.removeAll(c2);
        System.out.println("After removeAll: " + c);

        // clear()
        c.clear();
        System.out.println("After clear: " + c);

        // isEmpty() after clear
        System.out.println("Is empty: " + c.isEmpty());
    }
}
```

## Sample Output

```
After add: [10, 20, 30]
After addAll: [10, 20, 30, 40, 50]
After remove: [10, 30, 40, 50]
Contains 30: true
Contains all c2: true
Size: 4
Is empty: false
Elements using Iterator: 10 30 40 50
After removeAll: [10, 30]
After clear: []
Is empty: true
```

## Program 2: List Interface Demo

### Class: ListInterfaceDemo

```
import java.util.ArrayList;
import java.util.List;
import java.util.Comparator;

public class ListInterfaceDemo {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.add(30);
        list.add(10);
        list.add(20);
        list.add(10);
        System.out.println("List: " + list);

        list.add(1, 50);
        System.out.println("After adding 50 at index 1: " + list);

        System.out.println("Element at index 2: " + list.get(2));

        list.set(2, 100);
        System.out.println("After replacing index 2: " + list);

        list.remove(2);
        System.out.println("After removing index 2: " + list);

        System.out.println("First index of 10: " + list.indexOf(10));

        System.out.println("Last index of 10: " + list.lastIndexOf(10));

        System.out.println("SubList: " + list.subList(1, 3));

        list.sort(Comparator.naturalOrder());
        System.out.println("Sorted List: " + list);
    }
}
```

### Sample Output

```
List: [30, 10, 20, 10]
After adding 50 at index 1: [30, 50, 10, 20, 10]
Element at index 2: 10
After replacing index 2: [30, 50, 100, 20, 10]
After removing index 2: [30, 50, 20, 10]
First index of 10: 3
Last index of 10: 3
SubList: [50, 20]
Sorted List: [10, 20, 30, 50]
```

## Program 3: Set Interface Demo

### Class: SetInterfaceDemo

```
import java.util.HashSet;
import java.util.Set;

public class SetInterfaceDemo {
    public static void main(String[] args) {

        Set<Integer> set = new HashSet<>();

        // add(E e)
        set.add(10);
        set.add(20);
        set.add(30);
        set.add(20);
        System.out.println("Set: " + set);

        // remove(Object o)
        set.remove(20);
        System.out.println("After removing 20: " + set);

        // contains(Object o)
        System.out.println("Contains 10: " + set.contains(10));

        // size()
        System.out.println("Size of set: " + set.size());

        // isEmpty()
        System.out.println("Is set empty: " + set.isEmpty());

        // clear()
        set.clear();
        System.out.println("After clear: " + set);

        System.out.println("Is set empty: " + set.isEmpty());
    }
}
```

### Sample Output

```
Set: [20, 10, 30]
After removing 20: [10, 30]
Contains 10: true
Size of set: 2
Is set empty: false
After clear: []
Is set empty: true
```

## Program 4: SortedSet Interface Demo

### Class: SortedSetInterfaceDemo

```
import java.util.TreeSet;
import java.util.SortedSet;

public class SortedSetInterfaceDemo {
    public static void main(String[] args) {

        SortedSet<Integer> set = new TreeSet<>();

        set.add(50);
        set.add(20);
        set.add(40);
        set.add(10);
        set.add(30);

        System.out.println("Sorted Set: " + set);

        // first()
        System.out.println("First element: " + set.first());

        // last()
        System.out.println("Last element: " + set.last());

        // headSet()
        System.out.println("Head Set (< 30): " + set.headSet(30));

        // tailSet()
        System.out.println("Tail Set (>= 30): " + set.tailSet(30));

        // subSet()
        System.out.println("Sub Set (20 to 50): " + set.subSet(20, 50));

        // comparator()
        System.out.println("Comparator: " + set.comparator());
    }
}
```

### Sample Output

```
Sorted Set: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
Head Set (< 30): [10, 20]
Tail Set (>= 30): [30, 40, 50]
Sub Set (20 to 50): [20, 30, 40]
Comparator: null
```

## Program 5: NavigableSet Interface Demo

### Class: NavigableSetInterfaceDemo

```
import java.util.TreeSet;
import java.util.NavigableSet;

public class NavigableSetInterfaceDemo {
    public static void main(String[] args) {

        NavigableSet<Integer> set = new TreeSet<>();

        set.add(10);
        set.add(20);
        set.add(30);
        set.add(40);
        set.add(50);

        System.out.println("Navigable Set: " + set);

        // lower()
        System.out.println("Lower than 30: " + set.lower(30));

        // floor()
        System.out.println("Floor of 30: " + set.floor(30));

        // ceiling()
        System.out.println("Ceiling of 35: " + set.ceiling(35));

        // higher()
        System.out.println("Higher than 30: " + set.higher(30));

        // pollFirst()
        System.out.println("Removed first element: " + set.pollFirst());
        System.out.println("Set after pollFirst: " + set);

        // pollLast()
        System.out.println("Removed last element: " + set.pollLast());
        System.out.println("Set after pollLast: " + set);

        // descendingSet()
        System.out.println("Descending Set: " + set.descendingSet());
    }
}
```

### Sample Output

```
Navigable Set: [10, 20, 30, 40, 50]
Lower than 30: 20
Floor of 30: 30
Ceiling of 35: 40
Higher than 30: 40
Removed first element: 10
Set after pollFirst: [20, 30, 40, 50]
Removed last element: 50
Set after pollLast: [20, 30, 40]
Descending Set: [40, 30, 20]
```

## Program 6: Queue Interface Demo

### Class: QueueInterfaceDemo

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueInterfaceDemo {
    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();

        // add(E e)
        queue.add(10);
        queue.add(20);
        queue.add(30);
        System.out.println("Queue after add: " + queue);

        // offer(E e)
        queue.offer(40);
        System.out.println("Queue after offer: " + queue);

        // element()
        System.out.println("Head using element(): " + queue.element());

        // peek()
        System.out.println("Head using peek(): " + queue.peek());

        // remove()
        System.out.println("Removed using remove(): " + queue.remove());
        System.out.println("Queue after remove: " + queue);

        // poll()
        System.out.println("Removed using poll(): " + queue.poll());
        System.out.println("Queue after poll: " + queue);
    }
}
```

### Sample Output

```
Queue after add: [10, 20, 30]
Queue after offer: [10, 20, 30, 40]
Head using element(): 10
Head using peek(): 10
Removed using remove(): 10
Queue after remove: [20, 30, 40]
Removed using poll(): 20
Queue after poll: [30, 40]
```

## Program 7: Deque Interface Demo

### Class: DequeInterfaceDemo

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeInterfaceDemo {
    public static void main(String[] args) {

        Deque<Integer> deque = new ArrayDeque<>();

        // addFirst() and addLast()
        deque.addFirst(20);
        deque.addLast(30);
        System.out.println("After addFirst and addLast: " + deque);

        // offerFirst() and offerLast()
        deque.offerFirst(10);
        deque.offerLast(40);
        System.out.println("After offerFirst and offerLast: " + deque);

        // peekFirst() and peekLast()
        System.out.println("First element: " + deque.peekFirst());
        System.out.println("Last element: " + deque.peekLast());

        // removeFirst() and removeLast()
        System.out.println("Removed first: " + deque.removeFirst());
        System.out.println("Removed last: " + deque.removeLast());
        System.out.println("After removeFirst and removeLast: " + deque);

        // pollFirst() and pollLast()
        System.out.println("Polled first: " + deque.pollFirst());
        System.out.println("Polled last: " + deque.pollLast());
        System.out.println("After pollFirst and pollLast: " + deque);
    }
}
```

### Sample Output

```
After addFirst and addLast: [20, 30]
After offerFirst and offerLast: [10, 20, 30, 40]
First element: 10
Last element: 40
Removed first: 10
Removed last: 40
After removeFirst and removeLast: [20, 30]
Polled first: 20
Polled last: 30
After pollFirst and pollLast: []
```

## Program 8: Map Interface Demo

### Class: MapInterfaceDemo

```
import java.util.HashMap;
import java.util.Map;

public class MapInterfaceDemo {
    public static void main(String[] args) {

        Map<Integer, String> map = new HashMap<>();

        // put()
        map.put(101, "Ravi");
        map.put(102, "Sita");
        map.put(103, "Rahul");
        System.out.println("Map: " + map);

        // get()
        System.out.println("Value for key 101: " + map.get(101));

        // remove()
        map.remove(102);
        System.out.println("After removing key 102: " + map);

        // containsKey()
        System.out.println("Contains key 103: " + map.containsKey(103));

        // containsValue()
        System.out.println("Contains value Ravi: " + map.containsValue("Ravi"));

        // keySet()
        System.out.println("Keys: " + map.keySet());

        // values()
        System.out.println("Values: " + map.values());

        // entrySet()
        System.out.println("Entries: " + map.entrySet());

        // size()
        System.out.println("Size: " + map.size());

        // isEmpty()
        System.out.println("Is map empty: " + map.isEmpty());

        // clear()
        map.clear();
        System.out.println("After clear: " + map);

        System.out.println("Is map empty: " + map.isEmpty());
    }
}
```

### Sample Output

```
Map: {101=Ravi, 102=Sita, 103=Rahul}
Value for key 101: Ravi
After removing key 102: {101=Ravi, 103=Rahul}
Contains key 103: true
Contains value Ravi: true
Keys: [101, 103]
Values: [Ravi, Rahul]
Entries: [101=Ravi, 103=Rahul]
Size: 2
Is map empty: false
After clear: {}
Is map empty: true
```

## Program 9: SortedMap Interface Demo

### Class: SortedMapInterfaceDemo

```
import java.util.TreeMap;
import java.util.SortedMap;

public class SortedMapInterfaceDemo {
    public static void main(String[] args) {

        SortedMap<Integer, String> map = new TreeMap<>();

        map.put(103, "Rahul");
        map.put(101, "Ravi");
        map.put(105, "Arun");
        map.put(102, "Sita");
        map.put(104, "Priya");

        System.out.println("Sorted Map: " + map);

        // firstKey()
        System.out.println("First key: " + map.firstKey());

        // lastKey()
        System.out.println("Last key: " + map.lastKey());

        // headMap()
        System.out.println("Head Map (< 103): " + map.headMap(103));

        // tailMap()
        System.out.println("Tail Map (>= 103): " + map.tailMap(103));

        // subMap()
        System.out.println("Sub Map (102 to 105): " + map.subMap(102, 105));

        // comparator()
        System.out.println("Comparator: " + map.comparator());
    }
}
```

### Sample Output

```
Sorted Map: {101=Ravi, 102=Sita, 103=Rahul, 104=Priya, 105=Arun}
First key: 101
Last key: 105
Head Map (< 103): {101=Ravi, 102=Sita}
Tail Map (>= 103): {103=Rahul, 104=Priya, 105=Arun}
Sub Map (102 to 105): {102=Sita, 103=Rahul, 104=Priya}
Comparator: null
```

## Program 10: NavigableMap Interface Demo

### Class: NavigableMapInterfaceDemo

```
import java.util.TreeMap;
import java.util.NavigableMap;

public class NavigableMapInterfaceDemo {
    public static void main(String[] args) {

        NavigableMap<Integer, String> map = new TreeMap<>();

        map.put(101, "Ravi");
        map.put(102, "Sita");
        map.put(103, "Rahul");
        map.put(104, "Priya");
        map.put(105, "Arun");

        System.out.println("Navigable Map: " + map);

        // lowerKey()
        System.out.println("Lower key than 103: " + map.lowerKey(103));

        // floorKey()
        System.out.println("Floor key of 103: " + map.floorKey(103));

        // ceilingKey()
        System.out.println("Ceiling key of 103: " + map.ceilingKey(103));

        // higherKey()
        System.out.println("Higher key than 103: " + map.higherKey(103));

        // firstEntry()
        System.out.println("First entry: " + map.firstEntry());

        // lastEntry()
        System.out.println("Last entry: " + map.lastEntry());

        // pollFirstEntry()
        System.out.println("Removed first entry: " + map.pollFirstEntry());
        System.out.println("Map after pollFirstEntry: " + map);

        // pollLastEntry()
        System.out.println("Removed last entry: " + map.pollLastEntry());
        System.out.println("Map after pollLastEntry: " + map);

        // descendingMap()
        System.out.println("Descending Map: " + map.descendingMap());
    }
}
```

### Sample Output

```
Navigable Map: {101=Ravi, 102=Sita, 103=Rahul, 104=Priya, 105=Arun}
Lower key than 103: 102
Floor key of 103: 103
Ceiling key of 103: 103
Higher key than 103: 104
First entry: 101=Ravi
Last entry: 105=Arun
Removed first entry: 101=Ravi
Map after pollFirstEntry: {102=Sita, 103=Rahul, 104=Priya, 105=Arun}
Removed last entry: 105=Arun
Map after pollLastEntry: {102=Sita, 103=Rahul, 104=Priya}
Descending Map: {104=Priya, 103=Rahul, 102=Sita}
```

## Program 11: Iterator Interface Demo

### Class: IteratorInterfaceDemo

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorInterfaceDemo {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);
        list.add(50);

        System.out.println("List: " + list);

        Iterator<Integer> iterator = list.iterator();

        // hasNext() and next()
        System.out.println("Elements using Iterator:");

        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        // remove()
        iterator = list.iterator();

        while (iterator.hasNext()) {
            int value = iterator.next();

            if (value == 30) {
                iterator.remove();
            }
        }

        System.out.println("After removing 30: " + list);

        // forEachRemaining()
        iterator = list.iterator();

        System.out.println("Remaining elements:");

        iterator.forEachRemaining(value -> System.out.println(value));
    }
}
```

### Sample Output

```
List: [10, 20, 30, 40, 50]
Elements using Iterator:
10
20
30
40
50
After removing 30: [10, 20, 40, 50]
Remaining elements:
10
20
40
50
```

## Program 12: ListIterator Interface Demo

### Class: ListIteratorInterfaceDemo

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ListIteratorInterfaceDemo {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);

        System.out.println("List: " + list);

        ListIterator<Integer> iterator = list.listIterator();

        // hasNext(), next(), nextIndex()
        System.out.println("Forward Traversal:");

        while (iterator.hasNext()) {
            System.out.println("Index: " + iterator.nextIndex()
                               + ", Element: " + iterator.next());
        }

        // hasPrevious(), previous(), previousIndex()
        System.out.println("Backward Traversal:");

        while (iterator.hasPrevious()) {
            System.out.println("Index: " + iterator.previousIndex()
                               + ", Element: " + iterator.previous());
        }

        // add()
        iterator = list.listIterator();
        iterator.add(5);

        System.out.println("After add: " + list);

        // set()
        iterator = list.listIterator();
        iterator.next();
        iterator.set(15);

        System.out.println("After set: " + list);

        // remove()
        iterator = list.listIterator();
        iterator.next();
        iterator.remove();

        System.out.println("After remove: " + list);
    }
}
```

### Sample Output

```
List: [10, 20, 30, 40]
Forward Traversal:
Index: 0, Element: 10
Index: 1, Element: 20
Index: 2, Element: 30
Index: 3, Element: 40
```

```
Backward Traversal:  
Index: 3, Element: 40  
Index: 2, Element: 30  
Index: 1, Element: 20  
Index: 0, Element: 10  
After add: [5, 10, 20, 30, 40]  
After set: [15, 10, 20, 30, 40]  
After remove: [10, 20, 30, 40]
```